FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Fritz Mardaniel L. Sablan
Section:	AN42
Professor:	Ms. Crisola G. Tan
Date:	05/10/2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

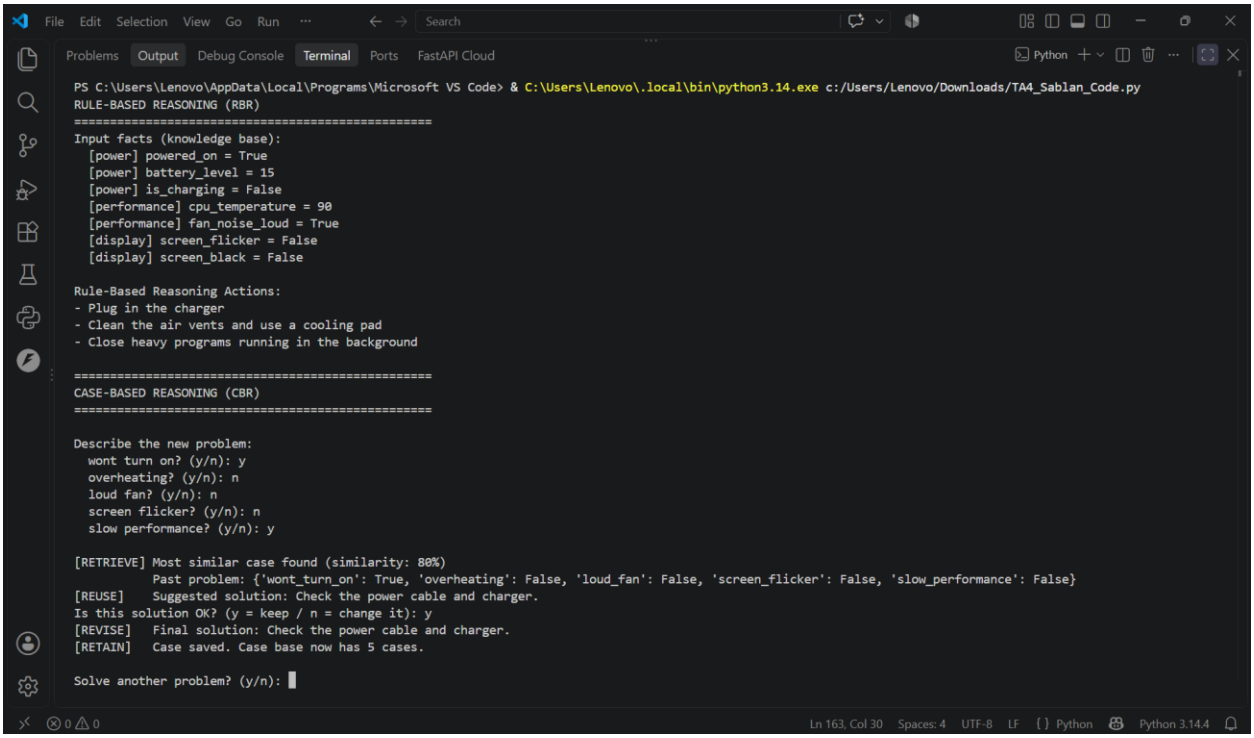
Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks ⇩

1. Rule-Based Reasoning was easier to implement because it uses simple if-then conditions. In the code, each rule checks a fact, such as whether the battery is below 20% or whether the CPU temperature is above 85°C, and then returns an appropriate action. The inference engine also simply checks each rule and collects the actions that match. CBR required more steps because it needed a case base, similarity calculation, retrieval of the most similar case, reuse of its solution, revision, and retention of the new case.
2. CBR is more flexible for solving unstructured problems because it can use previous cases that are similar to a new problem even when the new problem does not exactly match a rule.

3. Rule Based Reasoning can be better when the system receives continuously changing data. For example, the system can immediately evaluate changing values such as battery level and CPU temperature and produce action when a rule condition is met.
4. Examples for real-world applications for Rule Based Reasoning include laptop troubleshooting, medical decision-support systems, and simple diagnostic systems. Examples for CBR include customer support, technical troubleshooting, medical case analysis, and help-desk systems.




```
#Task 1A
facts = {
   #power-related facts
   "power": {
      "powered_on": True,
      "battery_level": 15, # percent
      "is_charging": False,
   },
   #performance-related facts
   "performance": {
      "cpu_temperature": 90, # degrees Celsius
      "fan_noise_loud": True,
   },
```

```
    #display-related facts
    "display": {
        "screen_flicker": False,
        "screen_black": False,
    },
}

#Task 2A
def rule_low_battery(facts):
    #If battery is below 20% and the laptop is not charging, then tell the user to plug in the charger
    if facts["power"]["battery_level"] < 20 and not facts["power"]["is_charging"]:
        return "Plug in the charger"

def rule_wont_turn_on(facts):
    #If the laptop is not powered on, then do a hard reset and try another outlet
    if not facts["power"]["powered_on"]:
        return "Hold the power button for 15 seconds and try a different outlet"

def rule_overheating(facts):
    #If the CPU temperature is above 85, then clean the vents and use a cooling pad
    if facts["performance"]["cpu_temperature"] > 85:
        return "Clean the air vents and use a cooling pad"

def rule_loud_fan(facts):
    #If the fan is loud, then close heavy programs
    if facts["performance"]["fan_noise_loud"]:
        return "Close heavy programs running in the background"

def rule_screen_flicker(facts):
    #If the screen is flickering, thenupdate the display driver
    if facts["display"]["screen_flicker"]:
        return "Update the display driver"

def rule_black_screen(facts):
     # If the screen is black while the laptop is on, then connect an external monitor to test the
display.
    if facts["power"]["powered_on"] and facts["display"]["screen_black"]:
        return "Connect an external monitor to test the display"

rules = [
    rule_low_battery,
    rule_wont_turn_on,
    rule_overheating,
    rule_loud_fan,
```

```
    rule_screen_flicker,
    rule_black_screen,
]

#Task 2B
def inference_engine(facts, rules):
    """Reads the knowledge base, checks each rule, returns all inferred actions."""
    actions = []
    for rule in rules:
        result = rule(facts)
        if result is not None:
            actions.append(result)
    return actions

#Task 2C
def run_rbr():
    print("=" * 50)
    print("RULE-BASED REASONING (RBR)")
    print("=" * 50)

    print("Input facts (knowledge base):")
    for category, items in facts.items():
        for name, value in items.items():
            print(f"  [{category}] {name} = {value}")

    inferred = inference_engine(facts, rules)

    print("\nRule-Based Reasoning Actions:")
    if inferred:
        for action in inferred:
            print(f"- {action}")
    else:
        print("- No rule matched. No action needed.")

#Task 3A
FEATURES = [
    "wont_turn_on",
    "overheating",
    "loud_fan",
    "screen_flicker",
    "slow_performance",
]

case_base = [
```

```
    {"problem": {"wont_turn_on": True, "overheating": False, "loud_fan": False,
                 "screen_flicker": False, "slow_performance": False},
     "solution": "Check the power cable and charger."},

    {"problem": {"wont_turn_on": False, "overheating": True, "loud_fan": True,
                 "screen_flicker": False, "slow_performance": True},
     "solution": "Clean the vents and use a cooling pad."},

    {"problem": {"wont_turn_on": False, "overheating": False, "loud_fan": False,
                 "screen_flicker": True, "slow_performance": False},
     "solution": "Update or reinstall the display driver."},

    {"problem": {"wont_turn_on": False, "overheating": False, "loud_fan": False,
                 "screen_flicker": False, "slow_performance": True},
     "solution": "Close background apps and free up disk space."},
]

#Task 3B
def similarity(new_problem, old_problem):
    """Fraction of features with the same value (0.0 = nothing alike, 1.0 = identical)."""
    matches = 0
    for feature in FEATURES:
        if new_problem.get(feature) == old_problem.get(feature):
            matches += 1
    return matches / len(FEATURES)

def retrieve(new_problem, case_base):
    """RETRIEVE: find the stored case most similar to the new problem."""
    best_case = None
    best_score = -1
    for case in case_base:
        score = similarity(new_problem, case["problem"])
        if score > best_score:
            best_case, best_score = case, score
    return best_case, best_score

#Task 3C
def reuse(best_case):
    """REUSE: use the solution of the most similar case."""
    return best_case["solution"]

#Task 3D
def revise(suggested_solution):
    """REVISE: let the student adjust the solution, or keep it unchanged."""
```

```
    answer = input("Is this solution OK? (y = keep / n = change it): ").strip().lower()
    if answer == "n":
        new_solution = input("Enter your revised solution: ").strip()
        if new_solution:
            return new_solution
    return suggested_solution

#Task 3E
def retain(new_problem, final_solution, case_base):
    """RETAIN: store the solved problem as a new case for future use."""
    case_base.append({"problem": new_problem, "solution": final_solution})

def ask_yes_no(question):
    """Helper: ask a yes/no question and return True or False."""
    while True:
        answer = input(f"{question} (y/n): ").strip().lower()
        if answer in ("y", "n"):
            return answer == "y"
        print("Please type y or n.")

def run_cbr():
    print("\n" + "=" * 50)
    print("CASE-BASED REASONING (CBR)")
    print("=" * 50)

    while True:
        print("\nDescribe the new problem:")
        new_problem = {}
        for feature in FEATURES:
            new_problem[feature] = ask_yes_no(f"  {feature.replace('_', ' ')}?")

        #1. RETRIEVE
        best_case, score = retrieve(new_problem, case_base)
        print(f"\n[RETRIEVE] Most similar case found (similarity: {score:.0%})")
        print(f"          Past problem: {best_case['problem']}")

        #2. REUSE
        suggestion = reuse(best_case)
        print(f"[REUSE]    Suggested solution: {suggestion}")

        #3. REVISE
        final_solution = revise(suggestion)
        print(f"[REVISE]   Final solution: {final_solution}")
```

```
        #4. RETAIN
        retain(new_problem, final_solution, case_base)
        print(f"[RETAIN]   Case saved. Case base now has {len(case_base)} cases.")

        if not ask_yes_no("\nSolve another problem?"):
            break

#MAIN
if __name__ == "__main__":
    run_rbr()
    run_cbr()
```